

PROYECTO INTEGRADOR · STREAMING DE DATOS Y SUS APLICACIONES

Consumo eléctrico por franja horaria, en streaming

Un pipeline end-to-end con Apache Kafka y Apache Beam sobre lecturas desordenadas, duplicadas y tardías

Sergio Morel Clara Almirón Daniel Ramírez

El dato que hace falta para facturar no existe

Una distribuidora eléctrica necesita cobrar la energía a precio distinto según la hora. Para eso hay que saber cuánto consumió cada cliente **en cada franja**. Y no se puede, por una razón física.

Bus compartido

Los medidores de una cabina comparten un bus RS-485 y se leen **de a uno, en secuencia**

Ronda de 6 a 60 min

Según cuántos medidores tenga la cabina y **cuántos fallen**

Nunca sobre el borde

Una lectura cae **en el medio** de la franja. Hay que repartir, y eso introduce error

Y ese error es plata: la energía se factura a precio distinto según la franja, así que una lectura atribuida a la franja equivocada se cobra mal.

EL GIRO

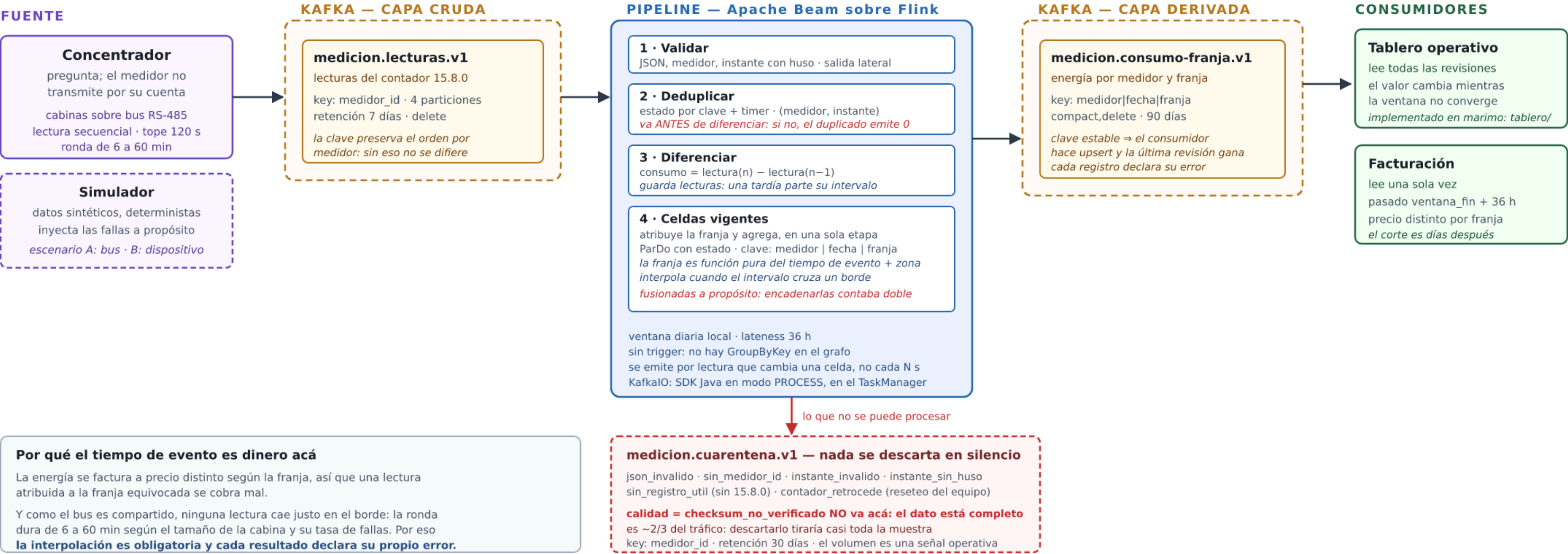
El sistema no entrega solo el consumo. Entrega cuánto se puede confiar en él.

Como los intervalos cruzan bordes de franja **siempre**, hay que repartir suponiendo potencia constante. Cada celda declara cuánto de ella se midió y cuánto se estimó.

Concentrador, log crudo, pipeline, log derivado

Consumo eléctrico por franja horaria — arquitectura

Proyecto integrador · Streaming de datos y sus aplicaciones · FPUNA. Flujo: concentrador → log crudo → pipeline → log derivado → consumidores.



El medidor entrega un contador acumulado (OBIS 15.8.0): el consumo no viene en el dato, se obtiene restando dos lecturas. Todos los datos son sintéticos.
Escenario B — un dispositivo de lectura por medidor, sin bus compartido — corre el MISMO pipeline y permite medir cuánto error elimina esa arquitectura.

Kafka 4.1.1 · Apache Beam 2.74 · Flink 1.19 como runner · KafkaIO corre el SDK de Java en modo PROCESS dentro del TaskManager

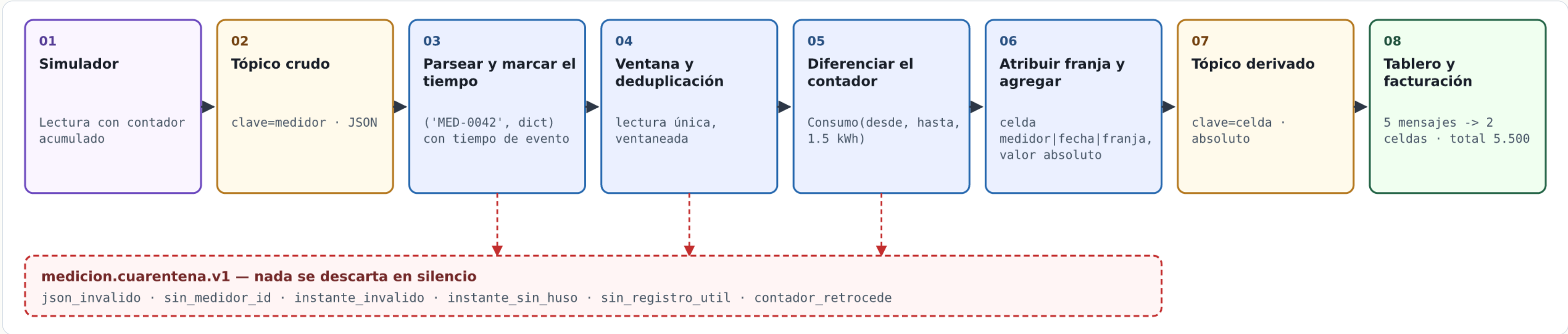
Cuatro decisiones que se defienden

Decisión	Por qué, y qué se descartó
La clave es el medidor , no la cabina	La cabina agruparía la ronda, pero van de 1 a 199 medidores: el reparto quedaría desbalanceado en un factor de 200
La franja no es una ventana	Es función pura del tiempo de evento: se calcula y viaja en la clave. Como ventana, cambiar el calendario obligaría a redefinir el ventaneo
Lateness de 36 horas	24 es la cadencia sin margen; 48 duplica el estado sin evidencia. 36 cubre un día de caída de enlace con media jornada de holgura
Umbral de 90 minutos para interpolar	No se eligió por gusto: la duración de los cruces tiene dos modas, y el umbral va en el valle que las separa

Son trece decisiones en total, cada una con su alternativa y lo que resigna

El recorrido de una lectura, del medidor al consumidor

MED-0042 · 25/09/2026 17:55 · contador en 101,5 kWh — la misma lectura de la demostración y del recorrido sobre Flink



Contador → consumo
101,5 kWh es el total histórico. El consumo aparece al restar dos lecturas (05).

Intervalo → celda
Un intervalo que cruza las 18:00 se reparte entre resto y punta, y se declara interpolado (06).

Medidor → celda como clave
La clave pasa a medidor|fecha|franja: eso hace idempotente la salida (07).

Cada estación, con el dato que entra y el que sale, está en el anexo del final

Tres escenarios, un solo medidor

Cinco lecturas alrededor de las 18:00, que es donde empieza **punta**. Cada acto se puede verificar con una resta mental.

Acto	Qué llega	Qué tiene que pasar
1	Tres lecturas normales	Dos intervalos; el segundo cruza el borde y se reparte por interpolación
2	Otra vez la de 17:55, idéntica	Nada cambia. Es un reintento de publicación, no una relectura
3	Tardía: 18:00, justo en el borde	Parte el intervalo y corrige el reparto que se había estimado

La tardía corrige el reparto sin cambiar el total

Celda	Actos 1 y 2	Acto 3	Error
... 2026-09-25 resto	2,300	2,400	+0,100
... 2026-09-25 punta	3,200	3,100	-0,100
TOTAL	5,500	5,500	0

Medir con más detalle **no crea ni destruye energía**: solo cambia a qué franja se le atribuye. Esos **0,100 kWh** son el error de atribución, y es el número que el proyecto existe para medir.

Y la columna de origen pasa de «interpolado» a «medido»: la lectura cayó sobre el borde y ya no hubo nada que estimar

Kafka y Flink de verdad, no un simulador de Beam

El stack

Kafka 4.1.1 en modo KRaft, con 4 particiones por tópico

Flink 1.19 con 2 TaskManagers de 2 slots, y el job server de Beam 2.74

KafkaIO corre el SDK de Java dentro del TaskManager

La prueba de humo

```
docker compose -f infra/docker-compose.yml  
--profile humo run --rm -T humo
```

Termina en **entraron 40 • salieron 40**. Prueba el cableado sin lógica de dominio: separa «el pipeline está mal» de «la infraestructura está mal».

Las mismas cinco lecturas, ahora sobre Kafka y Flink

Celda	Sobre Flink	Replay desde 0	Esperado
... 2026-09-25 punta	3,100	3,100	3,100
... 2026-09-25 resto	2,400	2,400	2,400
TOTAL	5,500	5,500	5,500

- Los mismos números** que con el runner local
- La cuarentena, vacía:** nada se perdió por el camino
- 5 mensajes para 2 celdas:** las revisiones; el upsert se queda con la última

Qué garantizamos, y dónde termina

Tramo	Garantía	Por qué
Concentrador → Kafka	Al menos una vez	acks=all e idempotencia. El reintento de publicación sí duplica, y lo absorbe la deduplicación
Dentro del pipeline	Efectivamente una vez , en 36 h y sin reinicios	Deduplicación con estado por medidor e instante
Pipeline → salida	Efectivamente una vez en el efecto	Upsert por clave estable y valor absoluto: reescribir es inocuo

No afirmamos exactly-once de punta a punta. Y el límite más serio: ningún checkpoint de Flink completa, así que **el estado no sobrevive a un reinicio**. Lo encontramos el día de la entrega; corregirlo invalidaba las seis corridas de evidencia.

La evidencia, en números

116

pruebas automáticas

Incluidas las de TestStream, las únicas que prueban lo tardío de forma determinista

6

corridas completas

En 4 máquinas distintas. Las seis dan 5,500 kWh con la cuarentena vacía

3

defectos, de afuera

Los encontró una persona ajena al equipo siguiendo solo el README

0,200

kWh que se mudan

En el tablero: el inyector manda una tardía y la energía pasa de resto a punta, con el total quieto

Una corrida exitosa con datos ideales no es evidencia

Lo dice el enunciado, y se cumplió **tres veces contra nuestro propio código**.

DOBLE CONTEO, DOS VECES

Un intervalo reemplazado que seguía sumando, y dos agregaciones encadenadas bajo modo acumulativo. Invisibles sin datos tardíos ni múltiples emisiones

LA PRUEBA EXTERNA

Una persona ajena al equipo levantó el sistema solo con el README. En quince minutos encontró un bug que **89 pruebas no vieron**: cubrían un punto de entrada, y el README usaba otro

CUANDO NO SE SABE

Si dos lecturas quedan a más de 90 minutos, ese cruce no se reparte: la celda se marca **indeterminada** y el resto se conserva. Preferimos decir «no sé» antes que inventar un número

Quién hizo qué

Sergio Morel

Simulador, infraestructura de Kafka y Flink, la cadena del pipeline y las franjas.

Encontró los dos errores de doble conteo

Clara Almirón

Contratos de evento, política temporal —ventana diaria y lateness de 36 h—, y el tablero en marimo con su inyector de irregularidades

Daniel Ramírez

Validación de reproducibilidad en una plataforma independiente: mismos resultados en otra máquina, y el hallazgo que derivó en el perfil de pruebas

El reparto exacto, escrito por cada uno, está en la sección 8 del documento técnico.

github.com/SEMP/streaming-medicion-tarifa-horaria

ANEXO • PARA LA DEFENSA

El recorrido de una lectura, estación por estación

Ocho láminas con el dato exacto que entra y que sale de cada etapa. No van en el video: están para responder preguntas.

Simulador

El dato nace porque alguien lo pidió

101,5 kWh no es lo consumido: es el total histórico del medidor. El consumo se obtiene restando dos lecturas.

El event_id sale del medidor y el instante. Un reintento de publicación produce el mismo id, y por eso el duplicado se reconoce.

LO QUE LLEGA

Nada. El medidor no transmite por su cuenta: el concentrador pregunta, medidor por medidor, sobre un bus RS-485 compartido.



QUÉ LE PASA

1. El concentrador recorre la cabina en orden
2. Pregunta el registro OBIS 15.8.0
3. El medidor responde su CONTADOR ACUMULADO, no el consumo del intervalo
4. Se arma la Lectura y se sella el event_id



LO QUE SALE — es lo que llega a la 02

```
{
  "schema_version": 1,
  "event_id": "d4612d921b39e34a",
  "medidor_id": "MED-0042",
  "cabina_id": "CAB-07",
  "instante_lectura": "2026-09-25T17:55:00-03:00",
  "registros": [{ "obis": "15.8.0",
    "naturaleza": "acumulado",
    "valor": 101.5, "unidad": "kWh" }],
  "calidad": "ok"
}
```

Tópico crudo

El log crudo: lo que llegó, tal como llegó

La clave es el medidor. Todas sus lecturas caen en la misma partición y se leen en orden: sin ese orden, restar no tiene sentido.

La cabina se descartó: van de 1 a 199 medidores, y el reparto entre particiones quedaría desbalanceado en un factor de 200.

LO QUE LLEGA — es la salida de la 01

```
{
  "schema_version": 1,
  "event_id": "d4612d921b39e34a",
  "medidor_id": "MED-0042",
  "cabina_id": "CAB-07",
  "instante_lectura": "2026-09-25T17:55:00-03:00",
  "registros": [{ "obis": "15.8.0",
    "naturaleza": "acumulado",
    "valor": 101.5, "unidad": "kWh" } ],
  "calidad": "ok"
}
```



QUÉ LE PASA

1. clave = medidor_id, en bytes
2. valor = ese mismo JSON, serializado en bytes
3. Kafka elige partición por hash de la clave
4. El productor va con acks=all e idempotencia



LO QUE SALE — es lo que llega a la 03

```
clave : b'MED-0042'
valor : b'{"schema_version": 1, "event_id":
        "d4612d921b39e34a", "medidor_id": ...}'

4 particiones • retención 7 días • cleanup.policy = delete
```

Parsear y marcar el tiempo

De bytes a un par con clave, y con el tiempo del dominio

Se usa cuándo se midió, no cuándo se publicó. Entre los dos puede haber horas, y ventanear por la publicación pondría el consumo en el día equivocado.

Un instante sin huso horario va a cuarentena. No se interpreta en la zona de la máquina donde corre el pipeline.

LO QUE LLEGA — es la salida de la 02

```
clave : b'MED-0042'  
valor : b'{"schema_version": 1, "event_id":  
        "d4612d921b39e34a", "medidor_id": ...}'
```



QUÉ LE PASA

1. `json.loads` del valor -> `('MED-0042', {...})`
el medidor sale del cuerpo; la clave es respaldo
2. Se lee `instante_lectura` y se exige su huso
3. Se sella como tiempo de EVENTO, no de llegada



LO QUE SALE — es lo que llega a la 04

```
TimestampedValue(  
    ('MED-0042', {...}),  
    timestamp = 1790369700  
)  
  
1790369700 = 2026-09-25 17:55:00-03:00  
{...} es el JSON de la estación 01, ya parseado a dict
```

Y SI NO SE PUEDE, VA A CUARENTENA

```
json_invalido · sin_medidor_id · instante_invalido · instante_sin_huso
```

Ventana diaria y deduplicación

El duplicado muere acá, antes de poder hacer daño

La ventana diaria se desplaza 3 horas. Sin eso, el día cortaría a medianoche UTC: las 21:00 en Asunción, en plena hora punta.

Deduplicar va antes de diferenciar. Al revés, el duplicado daría 0 kWh y, con salida por upsert, pisaría el valor bueno.

LO QUE LLEGA — es la salida de la 03

```
TimestampedValue(  
  ('MED-0042', {...}),  
  timestamp = 1790369700  
)
```



QUÉ LE PASA

1. Cae en la ventana del día local
FixedWindows(24 h), desplazada -3 h
2. ¿'2026-09-25T17:55:00-03:00' en VISTOS?
sí -> se cuenta y NO se emite
no -> se agrega y sigue
3. Timer para expirar a window.end + 36 h



LO QUE SALE — es lo que llega a la 05

```
TimestampedValue(  
  ('MED-0042', {...}),  
  timestamp = 1790369700  
) en la ventana del 2026-09-25 local  
  
estado VISTOS del medidor, después de esta lectura:  
{  
  '2026-09-25T17:40:00-03:00',  
  '2026-09-25T17:55:00-03:00'  
}
```

Y SI NO SE PUEDE, VA A CUARENTENA

instante_invalido

Diferenciar el contador

Recién acá aparece el consumo, que no venía en el dato

101,5 – 100,0 = 1,5 kWh en quince minutos. La lectura de las 17:40 estaba guardada en el estado del medidor.

Se guardan todas las lecturas, no solo la última: así una tardía puede partir en dos un intervalo ya emitido.

separacion_minutos es la cota del error de atribución, el número que el proyecto existe para medir.

LO QUE LLEGA — es la salida de la 04

```
TimestampedValue(  
  ('MED-0042', {...}),  
  timestamp = 1790369700  
) en la ventana del 2026-09-25 local
```



QUÉ LE PASA

1. Busca el registro OBIS 15.8.0 -> 101.5
2. Guarda (instante, valor, cabina) en el estado
3. Ordena TODAS las del medidor en la ventana
4. Arma los intervalos vecinos de esta lectura
5. energia = v_hasta - v_desde = 101.5 - 100.0



LO QUE SALE — es lo que llega a la 06

```
Consumo(  
  medidor_id='MED-0042',  
  cabina_id='CAB-07',  
  desde='2026-09-25T17:40:00-03:00',  
  hasta='2026-09-25T17:55:00-03:00',  
  energia_kwh=1.5,  
  separacion_minutos=15.0  
)  
  
la lectura de las 17:40, con el contador en 100.0,  
estaba guardada en el estado desde su propio paso por acá
```

Y SI NO SE PUEDE, VA A CUARENTENA

sin_registro_util · contador_retrocede

Atribuir la franja y agregar

Donde el intervalo se convierte en plata, y donde puede cruzar un borde

Un intervalo que cruza las 18:00 se reparte por minutos: 0,8 kWh a resto y 3,2 a punta, y las dos celdas se declaran interpoladas.

Se recalcula todo el medidor, no solo lo que tocó la lectura nueva: el resultado no depende del orden de llegada.

LO QUE LLEGA — es la salida de la 05

```
Consumo(  
  medidor_id='MED-0042',  
  cabina_id='CAB-07',  
  desde='2026-09-25T17:40:00-03:00',  
  hasta='2026-09-25T17:55:00-03:00',  
  energia_kwh=1.5,  
  separacion_minutos=15.0  
)
```



QUÉ LE PASA

1. ¿Hay ya un intervalo igual o más corto para ese borde izquierdo? Entonces este quedó superado y no se suma
2. Recalcula TODAS las celdas del medidor
3. Reparte por franja; si el intervalo cruza un borde, interpola suponiendo potencia constante, y lo declara
4. Emite solo lo que cambió



LO QUE SALE — es lo que llega a la 07

```
('MED-0042|2026-09-25|resto',  
{  
  "energia_kwh": 1.5, "interpolada": false,  
  "indeterminada": false, "minutos_cubiertos": 15.0,  
  "separacion_maxima_minutos": 15.0,  
  "cabina_id": "CAB-07", "intervalos_usados": 1})
```

y cuando llega el intervalo que cruza las 18:00,
esta misma celda se reemite y aparece la otra:

```
MED-0042|2026-09-25|resto 2.3 interpolada  
MED-0042|2026-09-25|punta 3.2 interpolada
```

Tópico derivado

El log derivado: la clave es la celda, no el evento

La clave pasa a ser la celda, y el valor es absoluto. Cada mensaje reemplaza al anterior en lugar de sumarse.

Reprocesar desde el offset 0 converge a las mismas celdas. Verificado sobre Kafka y Flink, no solo afirmado.

LO QUE LLEGA — es la salida de la 06

```
('MED-0042|2026-09-25|resto',  
{"energia_kwh": 1.5, "interpolada": false,  
 "indeterminada": false, "minutos_cubiertos": 15.0,  
 "separacion_maxima_minutos": 15.0,  
 "cabina_id": "CAB-07", "intervalos_usados": 1})
```



QUÉ LE PASA

1. clave = medidor|fecha|franja, en bytes
2. valor = el JSON de la celda, en bytes
3. KafkaIO Write, con el SDK de Java



LO QUE SALE — es lo que llega a la 08

clave : b'MED-0042|2026-09-25|resto'

valor : b'{"energia_kwh": 1.5, "interpolada":
false, ...}'

cleanup.policy = compact,delete • 90 días

la compactación por clave deja viva la última revisión de cada celda

Tablero y facturación

Dos lectores del mismo log, con necesidades opuestas

Cinco revisiones colapsan en dos celdas: $3,100 + 2,400 = 5,500$ kWh. Si el tablero sumara, el total daría más del doble.

El error de atribución lo calcula el consumidor: el denominador es la duración de la franja, que vive en el calendario.

LO QUE LLEGA — es la salida de la 07

```
clave : b'MED-0042|2026-09-25|resto'  
valor : b'{"energia_kwh": 1.5, "interpolada":  
        false, ...}'
```



QUÉ LE PASA

El tablero relee desde el offset 0 y hace upsert por clave: el último gana, nunca suma.

La facturación lee una sola vez, pasado ventana_fin + 36 h, cuando ya convergió.



LO QUE SALE

```
MED-0042|2026-09-25|punta 3.100 kWh  
MED-0042|2026-09-25|resto 2.400 kWh  
TOTAL 5.500 kWh  
error de atribución = separacion_maxima_minutos  
                      / duración de la franja
```